KuberSelfHeal: Diagnosing and Self-Healing Cross-Node Pod Connectivity Failures on a Raspberry Pi Kubernetes Cluster

Hem Tilva *24110374*
Vansh Barfiwala *24110380*
Pratham Choksi *24110275*
Shivangi Thaker *24110371*
Jils Shah *24110148*

I. Introduction

Kubernetes runs applications as pods spread over several machines (nodes). A request from one pod to another on a different node passes through many layers: the client resolves a Service name through the cluster DNS (CoreDNS), the Service selects backend pods by label, only Ready pods become endpoints, the Service forwards to a target port [1], and the packet finally crosses an overlay network (here Flannel [3], which uses VXLAN [4]) between the nodes. A mistake in any one of these layers produces the same symptom for the user: the application cannot reach its database.

Kubernetes already heals one kind of failure: it restarts crashed containers and recreates deleted pods, so the cluster converges to the state it was given. It does not correct a configuration that is valid but wrong. A Service whose selector has a typo, a workload scaled to zero, or a firewall rule that drops the tunnel is faithfully kept in place. Finding which layer is broken is left to an operator.

This project builds an agent that does this work for one concrete application: a backend pod on one node that reads from a PostgreSQL pod on another node, on a three-node k3s [2] cluster of Raspberry Pi 5 boards connected over a phone Wi-Fi hotspot. When the connection fails, the agent walks the path the connection takes, labels the cause with evidence, applies a fix only when it is safe and unambiguous, checks that the application really answers again, and undoes the change if it did not help. Any failure it does not cover is labelled *other* and handed to a human instead of being guessed at.

Contributions. (1) A reproducible testbed in which every request is forced across nodes, with a recorded healthy baseline. (2) A diagnosis agent that covers ten fault types in two groups, the Kubernetes service path and the cross-node network, and labels everything else *other*. (3) A healer with explicit safety rules: act only on a high-confidence label with an exact fix, verify by an HTTP 200, roll back, escalate. (4) A fault injector and an evaluation harness that inject each fault repeatedly in random order and score the labels and the healing. (5) Measurements on the real cluster: 24/24 and 18/18 correct labels for the two groups, and 30 of 33 injected faults healed with no change made in any no-fault or uncovered-fault run.

II. Problem Statement

Problem. Given a two-tier application whose backend (node A) cannot reach its database (node B), decide which layer of the connection is broken, and, where it is safe, restore the connection automatically.

Inputs. (a) The cluster state, read with *kubectl* (Deployments, pods, Services, EndpointSlices, events, ReplicaSets, nodes); (b) active tests run from toolbox pods: an HTTP request through the backend, TCP connects, ping (including do-not-fragment pings), iperf3, and, on the node itself, *ip route*, *iptables* and *tc*; (c) a healthy baseline recorded beforehand (loss, RTT, path MTU, throughput).

Outputs. One label from a fixed set of twelve: *healthy*, *other*, or one of the ten covered faults (Table I), with a confidence level, the evidence, a suggested fix and, when the fix is unambiguous, a machine-readable action. The healer then reports one outcome: *healthy* (nothing to do), *healed*, *escalated* (a human must decide) or *failed* (the fix did not help and was undone).

Assumptions and constraints. The backend and database run on different worker nodes; the agent runs on the control-plane node with *kubectl* access; the toolbox DaemonSets are deployed; one fault is present at a time. The nodes are connected only by Wi-Fi, which we also use to manage them, so injected network faults are applied only to the overlay (*flannel.1*, UDP 8472), never to the Wi-Fi interface, and each schedules its own undo after five minutes.

Scope. In scope are the ten faults of Table I. Everything else is out of scope and must be labelled *other*: for example a NetworkPolicy that blocks the database, the Wi-Fi link itself being down, a database-level error, or the diagnosis being unable to finish (API server unreachable). A label must hold in both directions: the agent reports a fault if and only if that fault is the cause.

Gap. The built-in controllers restore the state they were given but do not question it, and a generic error such as "connection refused" does not identify the layer. We found this in our first evaluation: k3s implements NetworkPolicy denials by rejecting packets, which the client sees as the same "connection refused" as a wrong port, and a naive rule mislabelled the policy as a port error. The agent therefore needs positive evidence for every label and a safe default for everything else.

III. Project Objectives

- **O1 Correct labels.** Label each injected covered fault correctly across repeated, randomly ordered trials; measured as accuracy per fault and as a confusion matrix.
- **O2 No false alarms.** Report *healthy* in every trial with no fault injected.
- **O3 Uncovered faults are** *other*. An injected fault outside the covered set (a NetworkPolicy) must be labelled *other*, never a covered label.
- **O4 Automatic healing.** Heal covered faults without human help, where "healed" means the application returns HTTP 200 again; measured as healed/injected and as time from the fix to a working application.
- **O5 Safety.** Make zero changes in no-fault and uncovered-fault trials, and undo any change that does not help.
- **O6 Speed.** Diagnose within seconds; measured as the mean and maximum diagnosis time per fault.

V. Architecture and Design

Fig. 1 shows the system. The control-plane Pi (*master*) runs the agent; the two worker Pis run the application, one tier each, so that every request crosses the network between them.

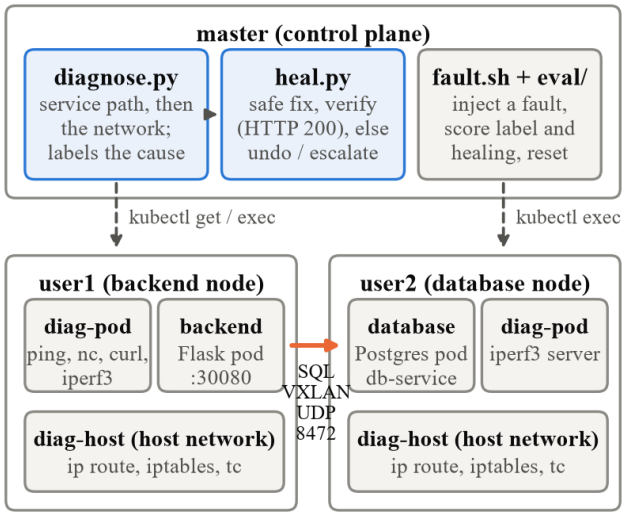


Fig. 1. System architecture. The agent on the control plane reads the cluster state and runs tests inside toolbox pods on the workers. The backend's database traffic crosses from user1 to user2 through the VXLAN tunnel.

- **Test application.** A Flask backend (NodePort 30080) on *user1* reads one row from PostgreSQL on *user2* through the Service *db-service* on every request; it is based on [8]. Node selectors pin the tiers to their nodes.
- **Toolbox pods.** *diag-pod* (on the pod network, so its traffic takes the same path as the application's) runs ping, TCP and HTTP tests and an iperf3 server; *diag-host* (privileged, host network) reads and changes the node's routes, firewall and traffic control.
- **Prober and baseline.** The prober requests the backend continuously and records each failure with a first guess of its layer. The baseline collector measures the healthy cluster: node to node, pod to pod across nodes and on the same node.
- **Diagnosis agent** (*diagnose.py*, *network.py*). Group 1 follows the connection path and stops at the first broken link: DB_HOST → DNS → Service → selector → Ready endpoints → target port. Group 2 runs only if group 1 is fine and compares layers: the Wi-Fi link between the nodes (underlay), the pod-to-pod tunnel (overlay) and the baseline.
- **Healer** (*heal.py*). Diagnose, apply one fix, verify, undo or escalate (Algorithm 2).
- **Fault injector and evaluation** (*fault.sh*, *run_diagnose.py*, *run_heal.py*). Inject a fault, wait, diagnose or heal, score, reset, and wait until the cluster is healthy again.

Table I. Covered faults, how they are injected, and the automatic fix

Label	Injected as	Automatic fix
dns_coredns_down	CoreDNS scaled to 0	scale CoreDNS to 1
wrong_service_name	backend DB_HOST = db-svc	set DB_HOST to the one similar Service that exposes the port
selector_mismatch	Service selector app=databse	patch the selector to the clearly closest pod labels
no_ready_endpoints	DB scaled to 0; readiness probe always fails	scale back; roll back a revision that never became Ready
port_mismatch	targetPort 5432 → 5433	patch targetPort to the port that is open
overlay_blocked	iptables DROP UDP 8472 on DB node	delete that rule
route_missing	pod-network route deleted on backend node	restore the flannel.1 route
mtu_blackhole	drop overlay packets > 1200 B	TCP MSS clamping to the measured path MTU
packet_loss	tc netem [7] loss 40%	remove the netem rule
high_latency	tc netem delay 150 ms	remove the netem rule

A. Design decisions and trade-offs

- **Follow the path, first broken link wins.** The checks run in the order the connection uses the layers, so a symptom further down (no endpoints) is not blamed when the cause is earlier (no DNS).
- **Positive evidence for every label, otherwise** *other*. A wrong port is reported only if the target port is not declared by the pod or another declared port is open. A selector is called wrong only if a pod exists that it was clearly meant to match (label similarity ≥ 0.6, e.g. app=databse vs. app=database scores 0.93) or it matches the backend itself; if no such pod exists the database is missing, which is *other*. *healthy* requires that the application was actually seen answering.
- **Compare layers and the baseline.** Pod-to-pod loss with clean node-to-node pings points at the tunnel, not the

Wi-Fi. The tunnel is tested against the toolbox pod on the database node, not the database pod, so a NetworkPolicy on the database cannot look like a broken tunnel. Loss and latency are confirmed with a second sample, so a short hotspot stall is not a fault.

- **Conservative healing.** Act only on high confidence with an exact action; one change at a time; verify by HTTP 200; undo a change that did not help, except a fix that removed a faulty rule, route or qdisc (undoing it would recreate the fault); wait for young pods before rolling them back; log every step. The trade-off is coverage: ambiguous cases, and loss or latency without a configuration cause (on Wi-Fi most likely physical), are escalated rather than fixed.
- **Application design for diagnosability.** The backend's health probes check only its own process, not the database; otherwise a network fault would remove the backend from its Service and hide the database error that identifies the layer. The backend runs threaded so its health check still answers while a request waits on the database. The database uses the Recreate strategy so a failing readiness probe really leaves no Ready endpoint.
- **Mitigation for MTU.** The path MTU cannot be raised from inside the cluster, so the fix clamps the TCP segment size to the measured path. The fault is a black hole: packets that are too big vanish without the ICMP message that path MTU discovery relies on [5], [6].

VI. Implementation

The agent, healer, prober, baseline collector and evaluation scripts are written in Python 3 using only the standard library; they call *kubectl* and run commands in the toolbox pods with *kubectl exec*. Fault injection is a Bash script. The backend uses Flask, psycopg2 and gunicorn (gthread worker, 4 threads) on a *python:3.11-slim* image built on the Pi; the database is *postgres:15-alpine*; the toolbox pods use *nicolaka/netshoot:v0.13* [9] (ping, nc, curl, iperf3, ip, iptables, tc). Algorithms 1 and 2 summarise the two programs.

Algorithm 1: Diagnose(cluster)
symptom ← HTTP GET backend via its ClusterIP (2 tries)
if CoreDNS has no ready pod **then** finding dns_coredns_down
svc ← Service named by DB_HOST
if svc missing **then** finding wrong_service_name; **stop**
pods ← pods selected by svc
if pods = ∅ **or** only the backend itself **then**
  if a workload template matches **then** no_ready_endpoints
  elif a similar pod exists **then** selector_mismatch
  else evidence for other; **stop**
if no Ready endpoint **then** no_ready_endpoints; **stop**
if targetPort refused **and** a declared port is open
  then port_mismatch; **stop**
Network(): underlay ping; overlay ping to diag-pod
  overlay ≥ 90% loss: check route, firewall, netem
  loss/RTT ≫ baseline (2 samples): loss | latency
  DF ping fails at baseline MTU: find path MTU; iperf3
if findings **then return** first finding
if symptom failed **then return** other (with evidence)
if symptom OK **then return** healthy **else return** other

Algorithm 2: Heal()
loop
  r ← Diagnose()
  if r = healthy **then return** healed / healthy
  if r = other **or** confidence ≠ high **or** no exact action
    then return escalated (with the evidence)
  apply r.action (one change); log it
  wait until HTTP 200, a different fault, or 150 s
  if same fault after 150 s **then** undo (unless it removed
    a faulty rule); **return** failed
  if ≥ 3 fixes **then return** escalated

A. Challenges and how they were solved

- **"Refused" is not a wrong port.** In the first evaluation all three NetworkPolicy trials were labelled *port_mismatch*, because k3s rejects denied traffic. The port check now requires positive evidence (Section V-A); the second run labelled all three *other*.
- **Rolling back a bad revision.** In the first healing run the healer escalated all three readiness faults instead of rolling them back: the rule trusted the Deployment's "Progressing" condition, which with the Recreate strategy can report the rollout as finished although the new pods never became Ready. The rule now looks at the pods and rolls back only when the current revision's pods have never been Ready since they started. This fix has not yet been re-run on the cluster.
- **Hotspot noise.** One ping burst can hit a momentary Wi-Fi stall, so loss and latency need a second sample and the MTU test must fail twice. A failed *kubectl exec* over the hotspot is treated as "unknown", never as a failed test.
- **Flannel does not restore a deleted route.** The route was still missing after 45 s, so this is a failure the platform does not heal by itself.
- **Truncated measurements.** iperf3 killed mid-transfer leaves incomplete JSON; the agent detects an established-but-stalled transfer from the partial output.
- **Unexpected errors.** Any error during diagnosis (API unreachable, *kubectl* failure) yields *other* with low confidence; the healer then stops, escalates and still writes its log.

Code repository: https://github.com/hemtilva/KuberSelfHeal

VII. Testbed Setup

Table II. Testbed

Item	Configuration
Nodes	3× Raspberry Pi 5, 8 GB RAM (quad-core Arm Cortex-A76); no GPU used
Roles	master: control plane and agent; user1: backend; user2: database
OS / Kubernetes	Debian 13 (arm64); k3s v1.36 with Flannel VXLAN (UDP 8472)
Network	phone Wi-Fi hotspot, 10.206.185.0/24 on wlan0
Pod networks	10.42.0.0/24 (master), 10.42.2.0/24 (user1), 10.42.1.0/24 (user2)
MTU	wlan0 1500; flannel.1 and cni0 1450 (50 B VXLAN overhead)
Workload	Flask backend (NodePort 30080) → PostgreSQL 15 (db-service:5432), 1 replica each

Agent settings	verify timeout 150 s; grace 45 s; max 3 fixes; loss fault > max(15%, base+10%); latency fault > base+50 ms and > 2× base; selector similarity ≥ 0.6
Evaluation	3 rounds, faults in random order, 20 s between inject and diagnose/heal, reset and wait for healthy after each trial

Table III. HEALTHY BASELINE (USER1 → USER2)

Path	Loss	RTT avg	Path MTU	Throughput
node → node	0%	10.2 ms	1500	14.9 Mbit/s
pod → pod, cross	0%	12.7 ms	1450	13.2 Mbit/s
pod → DB pod	0%	11.3 ms	–	–
pod → pod, same	0%	0.06 ms	–	–

The baseline also recorded DNS lookups (20/20, mean 5.4 ms) and 60 application requests (all successful; p50 95.3 ms, p95 118.0 ms, p99 161.9 ms). The Wi-Fi signal was −45 to −49 dBm on the three nodes.

VIII. EVALUATION

Each fault was injected three times in random order together with three no-fault trials (*none*); group 1 also includes the uncovered *netpol* fault. Detection and healing were scored in separate runs. All runs were on 5 Nov 2026 on the testbed of Section VII.

Table IV. DETECTION: CORRECT LABELS PER FAULT

Fault	Run 1	Run 2	Mean time
none (→ healthy)	3/3	3/3	2.0 s
coredns	3/3	3/3	3.1 s
wrong-name	3/3	3/3	2.7 s
selector	3/3	3/3	3.0 s
scale0	3/3	3/3	2.9 s
readiness	3/3	3/3	3.0 s
port	3/3	3/3	3.3 s
netpol (→ other)	0/3	3/3	3.3 s
Group 1 total	**21/24**	**24/24**	–
none (→ healthy)	–	3/3	7.0 s
fw-vxlan	–	3/3	19.4 s
route	–	3/3	18.3 s
mtu	–	3/3	20.6 s
loss	–	3/3	10.7 s
delay	–	3/3	8.7 s
Group 2 total	–	**18/18**	–

Detection (Table IV). For group 1, run 1 was the first version and run 2 the version after the "refused" fix; mean times are from run 2. Group 2 had a single run. In the final runs every fault, every no-fault trial and the uncovered NetworkPolicy received the correct label (24/24 and 18/18). Group-1 diagnosis takes about 3 s because it mostly reads Kubernetes objects; group 2 takes 7–21 s because it measures the network: 30-packet ping samples, a binary search of the path MTU and an iperf3 transfer.

Table V. HEALING OUTCOME PER FAULT

Fault	Outcome	Correct	Fix → OK
none	healthy (no change)	3/3	–
netpol	escalated (no change)	3/3	–
coredns	healed	3/3	5.4 s
wrong-name	healed	3/3	7.5 s
selector	healed	3/3	2.0 s
scale0	healed	3/3	8.1 s
port	healed	3/3	2.0 s
readiness	escalated	0/3	–
none (group 2)	healthy (no change)	3/3	–
fw-vxlan	healed	3/3	7.3 s
route	healed	3/3	7.3 s
loss	healed	3/3	7.3 s
delay	healed	3/3	7.3 s
mtu	healed	3/3	22.9 s

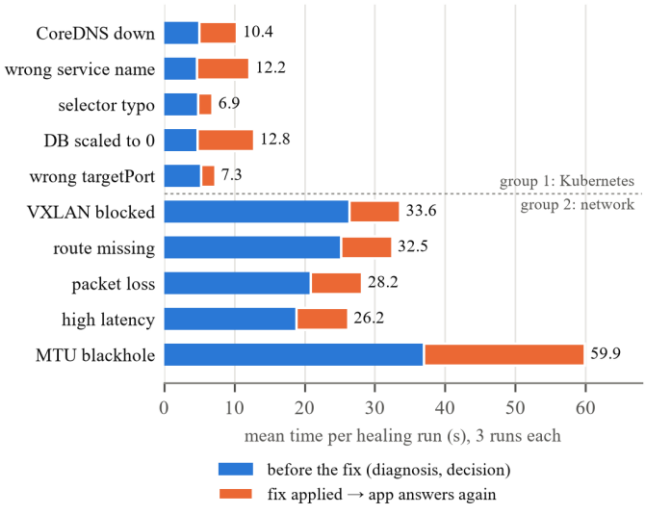


Fig. 2. Mean duration of a healing run per healed fault, split into the time before the fix (diagnosis and decision) and the time from the fix until the application answers HTTP 200 again.

Healing (Table V, Fig. 2). Of 33 injected covered faults, 30 were healed and the application answered again; an independent diagnosis after each run confirmed it. In all nine no-fault and NetworkPolicy trials the healer changed nothing, and no rollback was needed. The three misses are the readiness fault, which the first version escalated instead of rolling back (Section VI-A).

Kubernetes-level fixes bring the application back in 2–8 s on average; the longer cases include waiting for a pod rollout (scale0, wrong-name). For the network fixes, "fix → OK" is 7.3 s in every run because it ends with the verifying diagnosis, whose network tests take about 7 s; the actual recovery is at most that. The MTU fix takes 23 s because its verification repeats the MTU search and the bulk transfer: the small path MTU remains, but TCP now works across it. Most of a group-2 healing run is the diagnosis before the fix (19–37 s).

Validity. The faults are injected by us, so their signatures (one rule, one route, one qdisc) are clean; real faults may be messier. The results predate the last code changes (stricter labels, errors → *other*, the readiness rollback rule, safer undo and fix choice). These were checked in a simulated cluster (34/34 scenarios) but not yet re-run on the Pis.

IX. SUMMARY

We built an agent that explains and repairs a failed connection between two pods on different Kubernetes nodes. It follows the connection through service discovery and the overlay network, labels one of ten covered faults only with positive evidence, and labels everything else *other*. On a

Raspberry Pi 5 cluster over Wi-Fi it labelled every trial of the final runs correctly (24/24 and 18/18), healed 30 of 33 injected faults with the application answering again, and never changed anything when there was no fault or the fault was outside its scope.

Limitations. One application topology, one fault at a time, faults injected by us, and a single small cluster on one Wi-Fi network. Loss or latency without a configuration cause is only reported. NetworkPolicies are deliberately never changed. The readiness rollback and the latest label rules still need a run on the hardware. **Future work:** re-run the full evaluation on the current code; explain NetworkPolicy denials (which policy, which rule) without changing them; handle several simultaneous faults; test on larger clusters and other CNI plugins.

IX. Reproducibility

GitHub: https://github.com/hemtilva/KuberSelfHeal. All source code is in *code/*, whose README gives the full setup; in short:

Steps (on the control-plane Pi, inside code/, unless noted)
1. Install k3s with node-ip and flannel-iface = wlan0 on all Pis; label the workers role=backend and role=database.
2. On the backend Pi: build backend-api:latest from code/app and import it into k3s.
3. kubectl apply -f k8s/app.yaml
4. python3 baseline/collect.py (deploys k8s/diag.yaml, records the baseline)
5. python3 eval/run_diagnose.py --group all --rounds 3
6. python3 eval/run_heal.py --group all --rounds 3
Single fault: bash faults/fault.sh inject <fault>; python3 agent/diagnose.py; python3 agent/heal.py; bash faults/fault.sh reset

Each evaluation writes *trials.jsonl* and *report.txt* under *code/eval/results/*; the runs reported here are committed there.

References

[1] Kubernetes Documentation, “Services, Load Balancing, and Networking.” [Online]. Available: https://kubernetes.io/docs/concepts/services-networking/

[2] K3s Documentation. [Online]. Available: https://docs.k3s.io/

[3] Flannel. [Online]. Available: https://github.com/flannel-io/flannel

[4] M. Mahalingam et al., “Virtual eXtensible Local Area Network (VXLAN): A Framework for Overlaying Virtualized Layer 2 Networks over Layer 3 Networks,” RFC 7348, Aug. 2014.

[5] J. Mogul and S. Deering, “Path MTU Discovery,” RFC 1191, Nov. 1990.

[6] K. Lahey, “TCP Problems with Path MTU Discovery,” RFC 2923, Sep. 2000.

[7] S. Hemminger, “Network Emulation with NetEm,” in Proc. Linux Conf. Australia (LCA), 2005.

[8] P. Choksi, CN_Project_App. [Online]. Available: https://github.com/prathamchoksi/CN_Project_App

[9] nicolaka/netshoot. [Online]. Available: https://github.com/nicolaka/netshoot

APPENDIX A: TEAM WORK DISTRIBUTION AND MEMBER RATING

A.1 Work Distribution

#	Member Name	Roll No.	Tasks Done	Rating (1–10, unique)
1	Hem Tilva	24110374	Setting up the Raspberry Pi cluster (k3s on the hotspot, networking, toolbox pods); running the evaluations on the cluster; repository	8.5
2	Vansh Barfiwala	24110380	Background research and references; presentation slides and documentation	6.5
3	Pratham Choksi	24110275	Test application (Flask backend + PostgreSQL) and its arm64 Docker image; design of the self-healing approach (diagnose, fix, verify, undo)	10
4	Shivangi Thaker	24110371	Self-healing logic and safety rules (labels, “other”, rollback); review of the diagnosis decisions; evaluation analysis and report	9.5
5	Jils Shah	24110148	Research on Kubernetes and overlay-network failures (DNS, Services, VXLAN, MTU); fault-injection scenarios and healthy baseline	7.5

APPENDIX B: AI USAGE DETAILS

B.1 Prompts

#	AI Tool	Purpose
1	Claude Code (Claude Opus 5.5)	Writing the code: diagnosis agent, healer, fault injection, baseline and evaluation scripts (co-author of every commit after the initial one)
2	Claude Code (Claude Opus 5.5)	Reviewing the healing decisions and label logic; fixing gaps (errors → other, label exactness); simulated tests
3	Claude Code (Claude Opus 5.5)	Repository clean-up and drafting this report
4		

Relevant prompt queries and response .md files are shared in the ai/ folder of the GitHub repository (https://github.com/hemtilva/KuberSelfHeal/tree/main/ai): the raw transcripts of every Claude Code session, including the review sub-agents.